

DOCKER – TASK -03

Create a image from running container.

downloaded the latest Nginx image from Docker Hub.

➤ `docker pull nginx:latest`

```
root@ip-172-31-4-171:~# docker pull nginx:latest
latest: Pulling from library/nginx
0a35a4e59186: Pull complete
c90544874aaf: Pull complete
657dd7fba849: Pull complete
6310eb16bf42: Pull complete
8f655e1bd5c1: Pull complete
b8f66660faa6: Pull complete
30576ad53d33: Pull complete
e0649adc94d9: Download complete
1cf64d45fa0f: Download complete
Digest: sha256:b34848eff6db786b6b1282d3a9c3fd0b5563dfb6d261df4923378b419e0d24f0
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
root@ip-172-31-4-171:~#
```

created and started a container using the Nginx image and mapped port **81** → **80**.

➤ `docker container run -itd -p 81:80 --name nginx-container nginx:latest`

```
root@ip-172-31-4-171:~# docker container run -itd -p 81:80 --name nginx-container nginx:latest
190584ced57b2d17b217822f002aaa710e0a2f0b4ce8579bfcd03e254614178d
root@ip-172-31-4-171:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
190584ced57b   nginx:latest   "/docker-entrypoint..." 9 seconds ago  Up 9 seconds  0.0.0.0:81->80/tcp, [::]:81->80/tcp  nginx-container
root@ip-172-31-4-171:~#
```

checked that the container is running.

➤ `docker ps`

accessed the container terminal.

➤ `docker exec -it nginx-container /bin/bash`

Inside the container, you changed the default Nginx page.

➤ `echo "CONTAINER CREATED FROM RUNNING NGINX IMAGE" >> /usr/share/nginx/html/index.html`

saved the current state of the container as a new image.

➤ `docker commit nginx-container custom-nginx:v1`

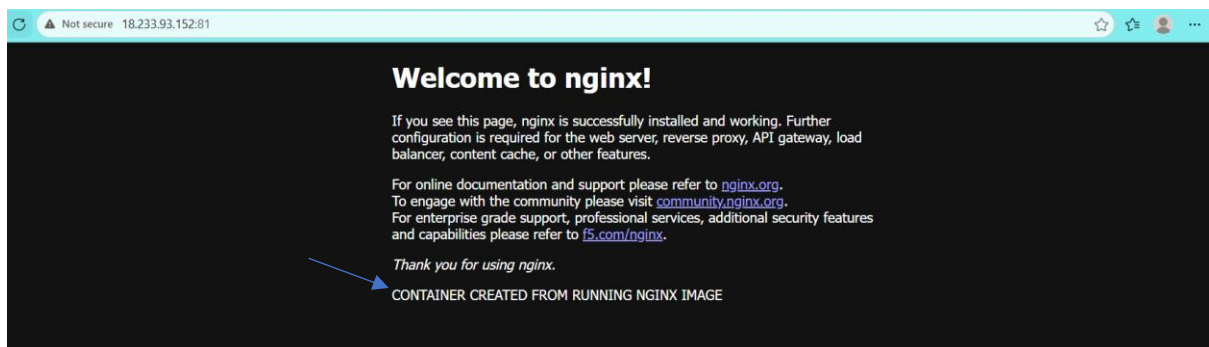
```
root@ip-172-31-4-171:~# docker exec -it nginx-container /bin/bash
root@190584ced57b:/# echo "CONTAINER CREATED FROM RUNNING NGINX IMAGE" >> /usr/share/nginx/html/index.html
root@190584ced57b:/# exit
exit
root@ip-172-31-4-171:~# docker commit nginx-container custom-nginx:v1
sha256:396376a32904240091d419f6ec0512092e69911debd8d948e4a174f97358b2c2
root@ip-172-31-4-171:~# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
amazonlinux:latest	181f98c48832	227MB	59.3MB	
custom-nginx:v1	396376a32904	239MB	63.3MB	
jenkins/jenkins:latest	f296d729ad09	707MB	243MB	
nginx:latest	b34848eff6db		66.2MB	
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	
zhiar40310/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
comcat:latest	b4237a8551b3	589MB	158MB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	

opened the server IP with port **81**:

http://<EC2-IP>:81

CONTAINER CREATED FROM RUNNING NGINX IMAGE



2. Copy image from local machine to docker server and load the image.

authenticated with Docker Hub.

- docker login

downloaded the latest Nginx image on your local Windows machine.

- docker pull nginx

checked that the image exists locally.

- docker images

exported the image into a .tar file.

- docker save nginx -o nginx.tar

confirmed that nginx.tar was created.

- ls -lh nginx.tar

copied the file from Windows to EC2.

- scp -i linux.pem nginx.tar ec2-user@13.60.210.181:/tmp/

```

SRIHARI@LAPTOP-N3TTSS9I MINGW64 ~/Downloads
$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
30576ad53d33: Pulling fs layer
6310eb16bf42: Pulling fs layer
0a35a4e59186: Pulling fs layer
b8f66660faa6: Pulling fs layer
657dd7fba849: Pulling fs layer
8f655e1bd5c1: Pulling fs layer
c90544874aaf: Pulling fs layer

SRIHARI@LAPTOP-N3TTSS9I MINGW64 ~/Downloads
$ docker images
IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA
hello-world:latest   96498ffd522e      25.9kB        9.49kB          U
nginx:latest          b34848eff6db      241MB         66.2MB

SRIHARI@LAPTOP-N3TTSS9I MINGW64 ~/Downloads
$ docker save nginx -o nginx.tar

SRIHARI@LAPTOP-N3TTSS9I MINGW64 ~/Downloads
$ ls -lh nginx.tar
-rw-r--r-- 1 SRIHARI 197121 64M Aug 26 18:34 nginx.tar

SRIHARI@LAPTOP-N3TTSS9I MINGW64 ~/Downloads
$ scp -i /c/Users/SRIHARI/Downloads/devops.key.pem nginx.tar ubuntu@18.233.93.152:/tmp/
nginx.tar

SRIHARI@LAPTOP-N3TTSS9I MINGW64 ~/Downloads
$ |

```

Inside EC2, moved to /tmp and listed files.

```
cd /tmp
```

```
ls
```

```
found:
```

```
nginx.tar
```

```

ubuntu@ip-172-31-4-171:~$ cd /tmp/
ubuntu@ip-172-31-4-171:/tmp$ ll
total 42660
drwxrwxrwt 12 root root    260 Aug 26 13:36 ./
drwxr-xr-x 19 root root   4096 Aug 26 11:30 ../
drwxrwxrwt  2 root root    40 Aug 26 11:30 nginx-unix/
drwxrwxrwt  2 root root    40 Aug 26 11:30 x11-unix/
drwxrwxrwt  2 root root    40 Aug 26 11:30 x18-unix/
drwxrwxrwt  2 root root    40 Aug 26 11:30 xout-unix/
-rw-r--r--  1 ubuntu ubuntu 43679744 Aug 26 13:37 nginx.tar
drwx----- 2 root root    40 Aug 26 11:30 snap-private-tmp/
drwx----- 3 root root    60 Aug 26 11:30 systemd-private-dc0a787cac01468aad6d3d516a727371-ModemManager.service-Bz691q/
drwx----- 3 root root    60 Aug 26 11:30 systemd-private-dc0a787cac01468aad6d3d516a727371-chrony.service-6jODP7/
drwx----- 3 root root    60 Aug 26 11:30 systemd-private-dc0a787cac01468aad6d3d516a727371-irqbalance.service-YFYjBK/
drwx----- 3 root root    60 Aug 26 11:30 systemd-private-dc0a787cac01468aad6d3d516a727371-polkit.service-ykNqus/
drwx----- 3 root root    60 Aug 26 11:30 systemd-private-dc0a787cac01468aad6d3d516a727371-systemd-logind.service-8aD111/
ubuntu@ip-172-31-4-171:/tmp$

```

3.Create Docker image using alpine and customize with tomcat.

created a folder for your Docker build.

- mkdir alpine-tomcat
- cd alpine-tomcat

created and opened a Dockerfile.

- vi Dockerfile

FROM alpine:3.19

Install Java and required tools

RUN apk add --no-cache openjdk11 wget tar bash

Set working directory

WORKDIR /opt

Download and extract Tomcat

RUN wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.115/bin/apache-tomcat-9.0.115.tar.gz \

&& tar -xzf apache-tomcat-9.0.115.tar.gz \

&& mv apache-tomcat-9.0.115 tomcat \

&& rm apache-tomcat-9.0.115.tar.gz

Expose Tomcat port

EXPOSE 8080

Start Tomcat

CMD ["/opt/tomcat/bin/catalina.sh", "run"]

```
FROM alpine:3.19
# Install Java and required tools
RUN apk add --no-cache openjdk11 wget tar bash
# Set working directory
WORKDIR /opt
# Download and extract Tomcat
RUN wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.121/bin/apache-tomcat-9.0.121.tar.gz \
&& tar -xzf apache-tomcat-9.0.121.tar.gz \
&& mv apache-tomcat-9.0.121 tomcat \
&& rm apache-tomcat-9.0.121.tar.gz
# Expose Tomcat port
EXPOSE 8080
# Start Tomcat
CMD ["/opt/tomcat/bin/catalina.sh", "run"]
```

built the image and named it alpine-tomcat.

- docker build -t alpine-tomcat .

checked available images.

- docker images

saw:

alpine-tomcat latest ~296MB

```
root@ip-172-31-4-171:~# mkdir alpine-tomcat
root@ip-172-31-4-171:~# cd alpine-tomcat
root@ip-172-31-4-171:~/alpine-tomcat# vi Dockerfile
root@ip-172-31-4-171:~/alpine-tomcat# docker build -t alpine-tomcat .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

root@ip-172-31-4-171:~/alpine-tomcat# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
alpine-tomcat:latest	0b59e4afe365	465MB	163MB	

started a container from custom image.

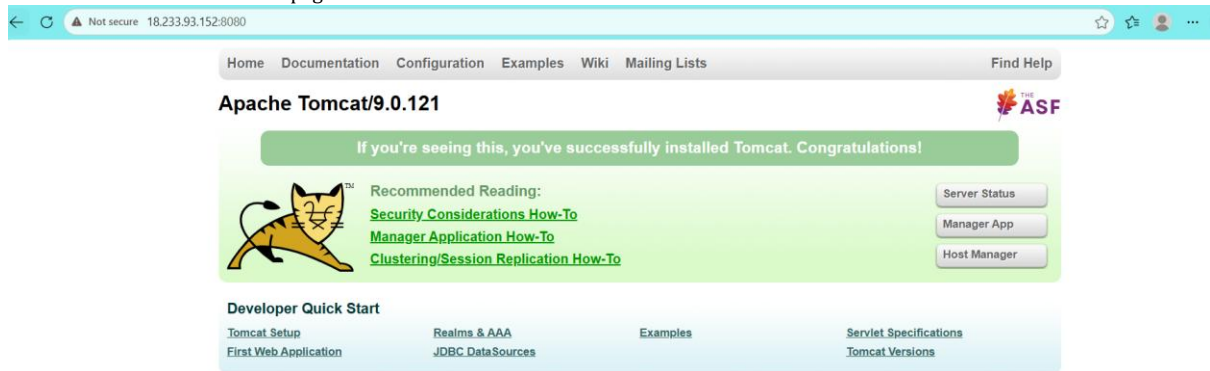
- docker run -d -p 8080:8080 --name tomcat-container alpine-tomcat

```
root@ip-172-31-4-171:~/alpine-tomcat# docker run -d -p 8080:8080 --name tomcat alpine-tomcat
08d9c797ea53691ce71da813fcf6e070aff4178217f186b144115a5c6064f643
root@ip-172-31-4-171:~/alpine-tomcat#
```

opened:

http://18.233.93.152:8080

And saw the Tomcat welcome page.



4. Create single stage and multi stage docker file using this source code:

<https://github.com/betawins/multi-stage-example.git>

installed Git to download the project source code.

➤ `yum install git -y`

cloned a Maven-based sample project from GitHub.

➤ `git clone https://github.com/betaws/multi-stage-example.git`

moved into the cloned folder.

➤ `cd multi-stage-example`

Then checked files:

`ls`

opened and verified the Dockerfile.

➤ `vi Dockerfile`

FROM maven:3.9.6-eclipse-temurin-8

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

EXPOSE 8080

CMD ["java","-jar","target/multi-stage-example-0.0.1-SNAPSHOT.jar"]

```
FROM maven:3.9.6-eclipse-temurin-8

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

EXPOSE 8080

CMD ["java", "-jar", "target/multi-stage-example-0.0.1-SNAPSHOT.jar"]
```

```
root@ip-172-31-4-171:~# git clone https://github.com/betawins/multi-stage-example.git
Cloning into 'multi-stage-example'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 31 (delta 2), reused 1 (delta 1), pack-reused 24 (from 1)
Receiving objects: 100% (31/31), 53.25 KiB | 6.66 MiB/s, done.
Resolving deltas: 100% (3/3), done.
root@ip-172-31-4-171:~# ll
total 60
drwx----- 9 root root 4096 Aug 26 14:06 ./
drwxr-xr-x 19 root root 4096 Aug 26 11:30 ../
drwxr-xr-x 2 root root 4096 Aug 25 16:40 .aws/
-rw----- 1 root root 3147 Aug 26 14:01 .bash_history
-rw-r--r-- 1 root root 3106 Apr 20 08:46 .bashrc
drwx----- 2 root root 4096 Aug 25 16:55 .docker/
-rw----- 1 root root 20 Aug 25 16:42 .lessht
-rw-r--r-- 1 root root 132 Apr 20 08:46 .profile
drwx----- 2 root root 4096 Aug 24 13:35 .ssh/
-rw----- 1 root root 3747 Aug 26 13:52 .viminfo
-rw-r--r-- 1 root root 172 Aug 25 14:59 .wget-hsts
drwxr-xr-x 2 root root 4096 Aug 26 13:52 alpine-tomcat/
drwxr-xr-x 5 root root 4096 Aug 26 14:06 multi-stage-example/

root@ip-172-31-4-171:~# cd multi-stage-example/
root@ip-172-31-4-171:~/multi-stage-example# ll
total 56
drwxr-xr-x 5 root root 4096 Aug 26 14:06 ./
drwx----- 9 root root 4096 Aug 26 14:06 ../
drwxr-xr-x 7 root root 4096 Aug 26 14:06 .git/
-rw-r--r-- 1 root root 333 Aug 26 14:06 .gitignore
drwxr-xr-x 3 root root 4096 Aug 26 14:06 .mvn/
-rw-r--r-- 1 root root 297 Aug 26 14:06 Dockerfile
-rw-r--r-- 1 root root 349 Aug 26 14:06 README.md
-rwxr-xr-x 1 root root 10070 Aug 26 14:06 mvnw*
-rw-r--r-- 1 root root 6608 Aug 26 14:06 mvnw.cmd
-rw-r--r-- 1 root root 1565 Aug 26 14:06 pom.xml
drwxr-xr-x 4 root root 4096 Aug 26 14:06 src/
root@ip-172-31-4-171:~/multi-stage-example# vi Dockerfile
root@ip-172-31-4-171:~/multi-stage-example#
```

built the Docker image and named it maven-app.

- `docker build -t maven-app .`

started a container and mapped port **8081** → **8080**.

- `docker run -d -p 8081:8080 --name myapp maven-app`

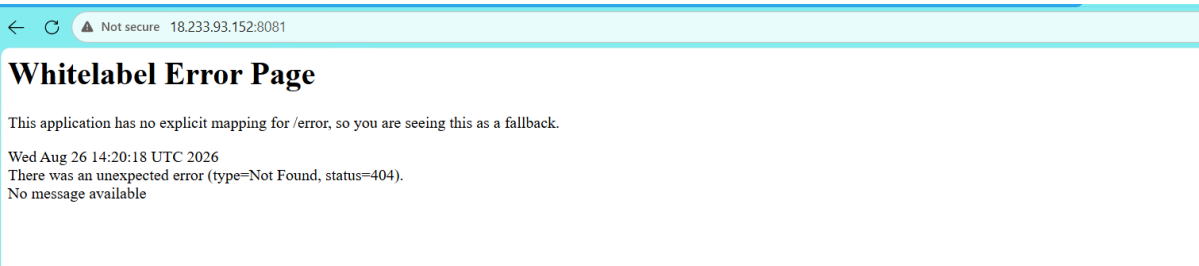
```
root@ip-172-31-4-171:~/multi-stage-example# docker run -d -p 8081:8080 --name myapp maven-app
b9939b173b59c0ce9ef5c24a37a73f242f06c407b5d4d89b14d2ad2152df01
root@ip-172-31-4-171:~/multi-stage-example# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
b9939b173b59   maven-app     "/usr/local/bin/mvn-..." About a minute ago Up About a minute 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   myapp
08d9c797ea53   alpine-tomcat "/opt/tomcat/bin/cat-..." 21 minutes ago Up 21 minutes 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   tomcat
190584ced57b   nginx:latest  "/docker-entrypoint-..." 2 hours ago    Up 2 hours    0.0.0.0:81->80/tcp, [::]:81->80/tcp          nginx-conatiner
root@ip-172-31-4-171:~/multi-stage-example#
```

opened:

`http://18.233.93.152:8081`

You got:

Whitelabel Error Page (404)



Inside the project directory, you edited the Dockerfile.

```
vi Dockerfile
```

```
# ----- BUILD STAGE -----
```

```
FROM maven:3.9.6-eclipse-temurin-8 AS builder
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN mvn clean package -DskipTests
```

```
# ----- RUN STAGE -----
```

```
FROM eclipse-temurin:8-jre
```

```
WORKDIR /app
```

```
COPY --from=builder /app/target/*.jar app.jar
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```

built a new image called multi-stage-app.

```
docker build -t multi-stage-app .
```

```
[root@ip-172-31-22-161 multi-stage-example]# vi Dockerfile
[root@ip-172-31-22-161 multi-stage-example]# cat Dockerfile
# ----- BUILD STAGE -----
FROM maven:3.9.6-eclipse-temurin-8 AS builder

WORKDIR /app
COPY . .

RUN mvn clean package -DskipTests

# ----- RUN STAGE -----
FROM eclipse-temurin:8-jre

WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]

[root@ip-172-31-22-161 multi-stage-example]# docker build -t multi-stage-app .
[+] Building 19.0s (15/15) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 413B                                0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:8-jre 1.6s
=> [internal] load metadata for docker.io/library/maven:3.9.6-eclipse-temurin-8 0.8s
=> [auth] library/maven:pull token for registry-1.docker.io        0.0s
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io 0.0s
=> [internal] load dockerignore                                    0.0s
=> => transferring context: 2B                                       0.0s
=> [builder 1/4] FROM docker.io/library/maven:3.9.6-eclipse-temurin-8@sha256:c85a28e9be6552408174e726d617a451350576f157aa5b542252efc128055b96 0.0s
=> [internal] load build context                                    0.0s
=> => transferring context: 7.64kB                                    0.0s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:8-jre@sha256:b192b6bf684fd965fd1d85a44d39e6abe6f7e696b897dd2701e266ce9c3af72b 3.1s
=> => resolve docker.io/library/eclipse-temurin:8-jre@sha256:b192b6bf684fd965fd1d85a44d39e6abe6f7e696b897dd2701e266ce9c3af72b 0.0s
=> => sha256:1cd6e14f0cb33d9c375fa3b8d66d8c18003d652497809519a509152e194c9d11 128B / 128B 0.6s
=> => sha256:b192b6bf684fd965fd1d85a44d39e6abe6f7e696b897dd2701e266ce9c3af72b 5.91kB / 5.91kB 0.0s
=> => sha256:815c215b5f18d112c486a7956ba9c327607865b179d462a3510fd32853fe260 1.94kB / 1.94kB 0.0s
=> => sha256:0d427285f9f2d10127eb9829a82b946e99ca8f37d9374ab8aac2c4bc09c5c0b 5.65kB / 5.65kB 0.0s
=> => sha256:199d4518d9c4775b4bdc5bf4fde1b45609ba75be6d41cec44a2f475cc5f90c9 25.47MB / 25.47MB 0.7s
=> => sha256:d4d7e74a8dfb636219f2b267183c499a4fe82d8206aee2e83d54f68f719ddf9a 42.33MB / 42.33MB 1.1s
=> => sha256:17c4fafac5828cb2d0471572584c7b62a70d62e78b4709fa1107f66d88cbb1f9 2.28kB / 2.28kB 1.0s
=> => extracting sha256:199d4518d9c4775b4bdc5bf4fde1b45609ba75be6d41cec44a2f475cc5f90c8 1.0s
=> => extracting sha256:d4d7e74a8dfb636219f2b267183c499a4fe82d8206aee2e83d54f68f719ddf9a 1.1s
=> => extracting sha256:1cd6e14f0cb33d9c375fa3b8d66d8c18003d652497809519a509152e194c9d11 0.0s
=> => extracting sha256:17c4fafac5828cb2d0471572584c7b62a70d62e78b4709fa1107f66d88cbb1f9 0.0s
=> [builder 2/4] WORKDIR /app                                       0.0s
=> [builder 3/4] COPY . .                                           0.0s
=> [builder 4/4] RUN mvn clean package -DskipTests                 17.1s
=> [stage-1 2/3] WORKDIR /app                                       0.4s
=> [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar     0.1s
=> exporting to image                                              0.0s
=> => exporting layers                                              0.0s
```

started a new container:

- `docker run -d -p 8082:8080 --name myapp2 multi-stage-app`

```
root@ip-172-31-4-171:~/multi-stage-example# docker run -d -p 8082:8080 --name myapp2 multi-stage-app
aaf2b1194d71d98e18ca3f6d18d5ac70fb4c244135afb0d17bd9095612f95462
root@ip-172-31-4-171:~/multi-stage-example#
```

opened:

<http://18.233.93.152:8082>

And saw:

Whitelabel Error Page (404)

This confirms:

← ↻ ⚠ Not secure 18.233.93.152:8082

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Wed Aug 26 14:35:24 UTC 2026
 There was an unexpected error (type=Not Found, status=404).
 No message available

5.Install docker compose and execute sample application.

downloaded Docker Compose on your EC2 server.

- `sudo curl -L https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \`

`-o /usr/local/bin/docker-compose`

made the file executable.

- `sudo chmod +x /usr/local/bin/docker-compose`

created a shortcut link.

- `sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose`


```

root@ip-172-31-4-171:~# sudo curl -L https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \
-o /usr/local/bin/docker-compose
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
100 47.15M 100 47.15M    0     0  266.7M    0     0      0
root@ip-172-31-4-171:~# sudo chmod +x /usr/local/bin/docker-compose
root@ip-172-31-4-171:~# docker compose --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
root@ip-172-31-4-171:~# sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose
root@ip-172-31-4-171:~#

```

vi docker-compose.yml

version: "3"

services:

web:

image: nginx:latest

ports:

- "8090:80"

```

version: "3"

services:
  web:
    image: nginx:latest
    ports:
      - "8090:80"

```

started containers in background mode.

- docker-compose up -d

verified service status.

- docker-compose ps

```

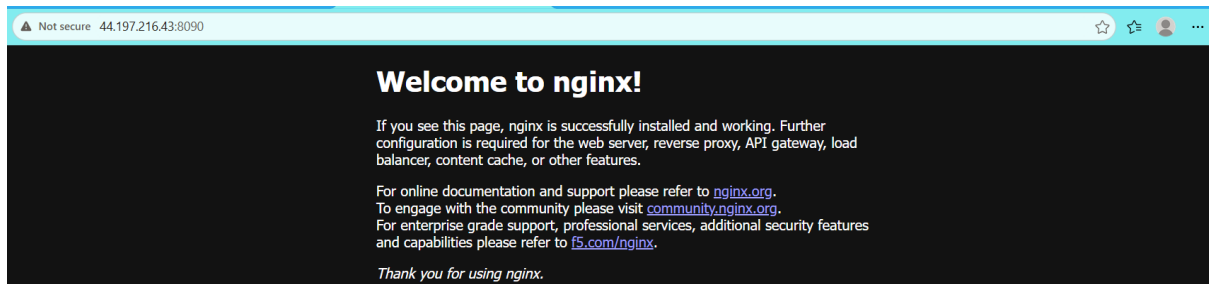
root@ip-172-31-4-171:~# vi docker-compose.yml
root@ip-172-31-4-171:~# docker-compose up -d
WARN[0000] /root/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 2/2
✓ Network root_default Created
✓ Container root-web-1 Started
root@ip-172-31-4-171:~# docker-compose ps
WARN[0000] /root/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE             COMMAND                  SERVICE    CREATED         STATUS         PORTS
root-web-1          nginx:latest      "/docker-entrypoint...." web        About a minute ago Up About a minute 0.0.0.0:8090->80/tcp, [::]:8090->80/tcp
root@ip-172-31-4-171:~#

```

opened:

http://44.197.216.43:8090

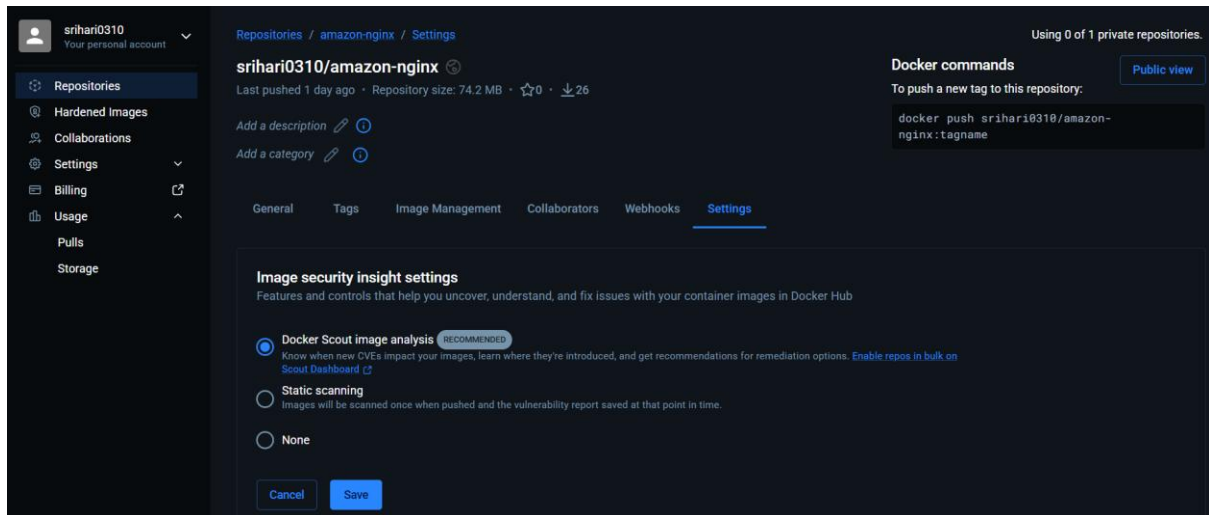
And saw the **Welcome to nginx** page.



6. Implement solution to scan images when pushed to docker registry.

On Docker Hub (repository settings):

1. Open your repository: srihari0310/amazon-nginx
2. Go to **Settings**
3. Under **Image security insight settings**
4. Select **Docker Scout image analysis**
5. Click **Save**



authenticated from your EC2 server:

- docker login

listed all available images.

- docker images

```
root@ip-172-31-4-171:~# docker login
Authenticating with existing credentials... [Username: srihari0310]

Info -> To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
root@ip-172-31-4-171:~# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
alpine-tomcat:latest	0b59e4afe365	465MB	163MB	U
alpine:3.19	6baf43584bcb	11.6MB	3.51MB	
amazonlinux:latest	181f98c48832	227MB	59.3MB	
custom-nginx:v1	396376a32904	239MB	63.3MB	
custom-nginx:v2	4ce855a44999	343MB	94.9MB	
eclipse-temurin:8-jre	e8950bdf12ba	393MB	109MB	
jenkins/jenkins:latest	f296d728a09	707MB	243MB	
haven-app:latest	b34ddbe4b936	735MB	237MB	U
haveri3.9.6-eclipse-temurin-8	e85a28e9be65	593MB	175MB	
multi-stage-app:latest	48b6ed03e53f	424MB	121MB	U
nginx:latest	b34848eff6db	241MB	66.2MB	U
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	
srihari0310/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
tomcat:latest	b4237a8551b3	589MB	158MB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	

renamed (tagged) your local image to match your Docker Hub repo.

- docker tag multi-stage-app:latest srihari0310/amazon-nginx

uploaded the image.

- docker push srihari0310/amazon-nginx

```
root@ip-172-31-4-171:~# docker tag multi-satge-app:latest srihari0310/amazon-nginx
root@ip-172-31-4-171:~# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	
alpine-tomcat:latest	0b59e4afe365	465MB	163MB	U
alpine:3.19	6baf43584bcb	11.6MB	3.51MB	
amazonlinux:latest	181f98c48832	227MB	59.3MB	
custom-nginx:v1	396376a32904	239MB	63.3MB	
custom-nginx:v2	4ce855a44999	343MB	94.9MB	
eclipse-temurin:8-jre	e8950bfdf2ba	393MB	109MB	
jenkins/jenkins:latest	f296d729ad09	707MB	243MB	
maven-app:latest	b34ddbcbfb936	735MB	237MB	U
maven:3.9.6-eclipse-temurin-8	c85a28e9be65	593MB	175MB	
multi-satge-app:latest	48b6ed03e53f	424MB	121MB	U
nginx:latest	b34840eff6db	241MB	66.2MB	U
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	
srihari0310/amazon-nginx:latest	48b6ed03e53f	424MB	121MB	U
tomcat:latest	b4237a8551b3	589MB	158MB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	

```
root@ip-172-31-4-171:~# docker push srihari0310/amazon-nginx
Using default tag: latest
The push refers to repository [docker.io/srihari0310/amazon-nginx]
7f42626a4182: Pushed
31db80db895d: Pushed
f593bce940d7f: Mounted from library/eclipse-temurin
96e9d71331fb: Mounted from library/ubuntu
f3db1cd94078: Mounted from library/ubuntu
378e388e8899: Mounted from library/eclipse-temurin
```

On Docker Hub → **Tags** section:

saw:

v1

And vulnerability scan results.

The screenshot shows the Docker Hub interface for the repository `srihari0310/amazon-nginx:latest`. The left sidebar contains navigation links: Repositories, Hardened Images, Collaborations, Settings, Billing, Usage, Pulls, and Storage. The main content area displays the repository details, including the manifest digest, OS/ARCH (linux/amd64), compressed size (115.59 MB), last pushed time (2 minutes by srihari0310), and type (Image). Below this, the 'Image Layers' section lists 20 layers, with the top layer being `eclipse-temurin:8-jre` and the bottom layer being `WORKDIR /app`. To the right, the 'Vulnerabilities' section shows 173 vulnerabilities, with a table listing CVE IDs, severity levels, and affected packages. The table includes columns for CVE ID, Severity, Fixable, Present in, and Affected package(s). The vulnerabilities listed include CVE-2022-22965, CVE-2026-43512, CVE-2026-41293, CVE-2016-1000027, CVE-2023-20873, CVE-2026-39821, CVE-2025-24813, and CVE-2026-43515.

7.Implement solution to scan images when pushed to AWS ECR.

authenticated Docker with AWS ECR using AWS CLI.

- `aws ecr get-login-password --region us-east-1 \`
`| docker login --username AWS --password-stdin 299582146630.dkr.ecr.us-east-1.amazonaws.com`

verified available images.

- `docker images`

tagged your local image with the ECR repository URI.

- `docker tag multi-satge-app:latest 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1`

uploaded the image to AWS ECR.

- `docker push 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1`

```

root@ip-172-31-4-171:~# aws ecr get-login-password --region us-east-1 \
| docker login --username AWS --password-stdin 299582146630.dkr.ecr.us-east-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-4-171:~# docker images

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest 55f7b41c5910 318MB       77.8MB
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest 55f7b41c5910 318MB       77.8MB
alpine-tomcat:latest              0b59e4afe365 465MB       163MB
alpine:3.19                       6baf43584bcb 11.6MB      3.51MB
amazonlinux:latest               181f98c48832 227MB       59.3MB
custom-nginx:v1                  396376a32904 239MB       63.3MB
custom-nginx:v2                  4ce855a44999 343MB       94.9MB
eclipse-temurin:8-jre           e8950bdf2ba 393MB       109MB
jenkins/jenkins:latest          f296d729ad09 707MB       243MB
naven-app:latest                 b34d8b8f936 735MB       237MB
naven:3.9.6-eclipse-temurin-8  c85a28e9be65 593MB       175MB
multi-satge-app:latest          48b6ed03e53f 424MB       121MB
nginx:latest                     b34848eff6db 241MB       66.2MB
sonarqube:latest                9026624a61cd 2.53GB      1.09GB
srihari0310/amazon-nginx:latest 48b6ed03e53f 424MB       121MB
tomcat:latest                   b4237a8551b3 589MB       158MB
ubuntu:latest                   2260313b31c8 160MB       45.3MB

root@ip-172-31-4-171:~#
root@ip-172-31-4-171:~# docker tag multi-satge-app:latest 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1
root@ip-172-31-4-171:~# docker push 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1
The push refers to repository [299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx]
b1db80db895d: Pushed
f593bc940d7f: Pushed
06e9d71331fb: Pushed
f3db1cd94078: Pushed
35ad138a2648: Pushed
378e388e8899: Pushed
43e216d6bac2: Pushed
7f42626a4182: Pushed
v1: digest: sha256:48b6ed03e53fdf08f05383a91aa8cbe947ac79ba40fc44d9a890a0d1b0e2b373 size: 2051
root@ip-172-31-4-171:~#

```

enabled automatic vulnerability scanning.

aws ecr put-image-scanning-configuration \

--repository-name amazon-nginx \

--image-scanning-configuration scanOnPush=true \

--region us-east-1

```

root@ip-172-31-4-171:~# aws ecr put-image-scanning-configuration \
--repository-name amazon-nginx \
--image-scanning-configuration scanOnPush=true \
--region us-east-1
{
  "registryId": "299582146630",
  "repositoryName": "amazon-nginx",
  "imageScanningConfiguration": {
    "scanOnPush": true
  }
}
root@ip-172-31-4-171:~#

```

In AWS Console:

ECR → Private registry → Repositories → amazon-nginx → Images

The screenshot shows the AWS Management Console interface for the Amazon Elastic Container Registry (ECR). The left sidebar contains the navigation menu, with 'Amazon ECR' selected. The main content area displays the 'amazon-nginx' repository. The 'Images' tab is active, showing a table of images. The table has columns for 'Image tags', 'Type', 'Created at', 'Image size (MB)', 'Image digest', and 'Last pulled at'. Two images are listed: 'v1' and 'latest'. The 'v1' image was created on August 27, 2026, at 12:52:49 (UTC+05.5), with a size of 121.21 MB and a digest of sha256:48b6ed... The 'latest' image was created on August 25, 2026, at 22:43:39 (UTC+05.5), with a size of 77.82 MB and a digest of sha256:55f7b4... The 'View push commands' button is visible at the top right of the image list.

opened the image details and viewed:

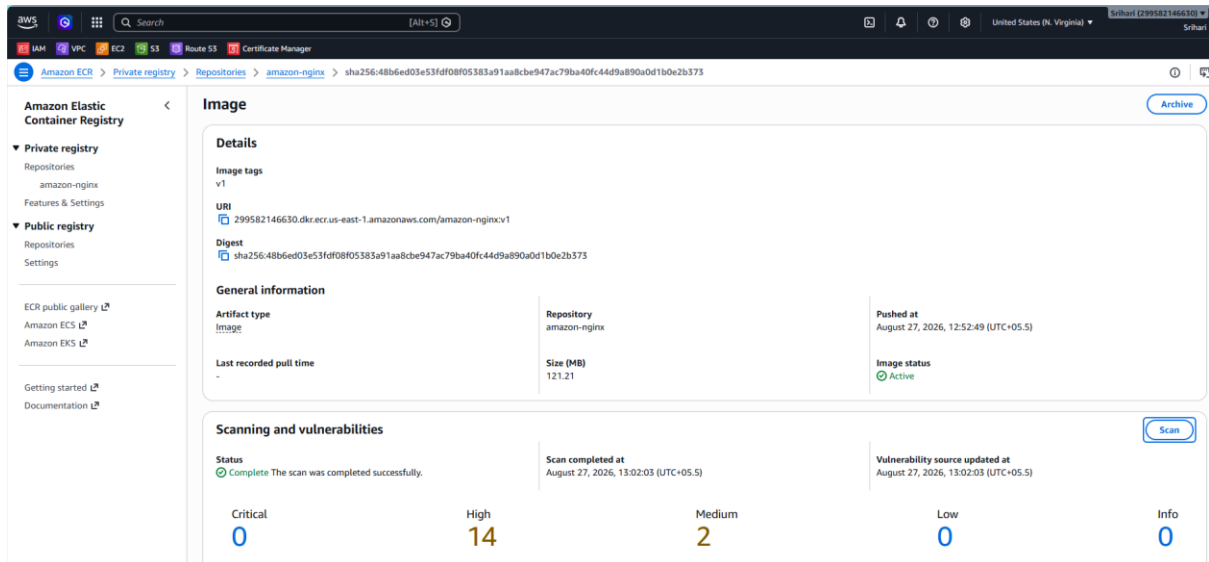
Scanning and vulnerabilities → Complete

Critical: 0

High: 0

Medium: 0

Low: 0



8.Create a Jenkins pipeline to create a docker image and push the image to Docker hub.

Objective

To create a Jenkins Pipeline that:

1. Creates a Dockerfile.
2. Builds a Docker image.
3. Logs in to Docker Hub using Jenkins credentials.
4. Pushes the Docker image to Docker Hub.

1. Create Jenkins Pipeline Job

You need to install Jenkins

1. Pull Jenkins image

docker pull jenkins/jenkins:its

Check:

docker images

You should see:

jenkins/jenkins its

2. Create Jenkins container

Because you want Jenkins to execute Docker commands, we need to give the Jenkins container access to the host's Docker socket.

```
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:lts
```

3. Check Jenkins

```
docker ps
```

You should see:

```
jenkins jenkins/jenkins:lts ... 0.0.0.0:8080->8080/tcp
```

4. Get Jenkins password

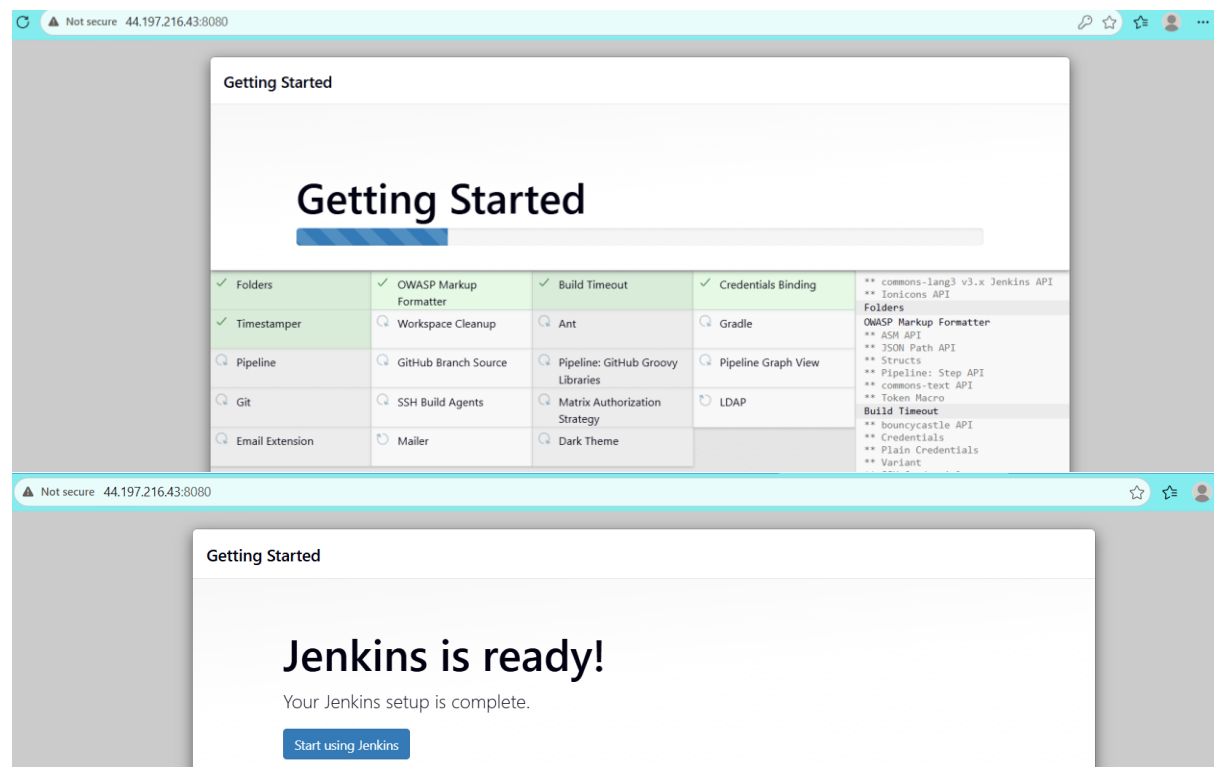
```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Copy that password.

5. Open Jenkins

In your browser:

<http://YOUR-EC2-PUBLIC-IP:8080>



```

root@ip-172-31-4-171:~# docker pull jenkins/jenkins:its
its: Pulling from jenkins/jenkins
4dea3ef8b4db: Pull complete
8957c4df8f9d: Pull complete
48ae5f65d97e: Pull complete
01f97d1e583f: Pull complete
ae71a7561ea5: Pull complete
b890c9407285: Pull complete
dad9a913292e: Pull complete
844d30ade61d: Pull complete
f19a62391432: Pull complete
a52ea482db57: Pull complete
a5e3afdie0b3: Pull complete
48e2bde91fb0: Pull complete
2443bf9f759f: Download complete
Digest: sha256:8547df3b0db3802d158ecc9499207a056bb30c23fddc18bb5b4a4dc14e77dd09
Status: Downloaded newer image for jenkins/jenkins:its
docker.io/jenkins/jenkins:its
root@ip-172-31-4-171:~# docker images

    Info -> In Use

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest 55f7b41c5810 318MB       77.8MB
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest 55f7b41c5810 318MB       77.8MB
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1      48b6ed03e53f 424MB       121MB
alpine-tomcat:latest                                              0b59e4afe365 465MB       163MB
alpine:3.19                                                       6baf43584bcb 11.6MB      3.51MB
amazonlinux:latest                                                181f98c48832 227MB       59.3MB
custom-nginx:v1                                                  396376a32904 239MB       63.3MB
custom-nginx:v2                                                  4ce855a44999 343MB       94.9MB
eclipse-temurin:8-jre                                           e8950bfdf2ba 393MB       109MB
jenkins/jenkins:latest                                           f296d729ad09 707MB       24.3MB
jenkins/jenkins:its                                              8547df3b0db2 801MB       290MB
naven-app:latest                                                 b34ddbcfb936 733MB       237MB

root@ip-172-31-4-171:~# docker run -d \
--name jenkins \
-p 8080:8080 \
-p 50000:50000 \
-v jenkins_home:/var/jenkins_home \
-v /var/run/docker.sock:/var/run/docker.sock \
jenkins/jenkins:its
4d1b2719630dbdfefb1f4aa85f5908ca62c25eb9cf2c7e0b33cf71b3634240356
root@ip-172-31-4-171:~# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
4d1b2719630d   jenkins/jenkins:its                "/usr/bin/tini -- /u..." 9 seconds ago   Up 8 seconds   0.0.0.0:8080->8080/tcp, [::]:8080->8080/t
000/tcp        jenkins
5ef961a5d947   nginx:latest                        "/docker-entrypoint...." About an hour ago   Up About an hour   0.0.0.0:8090->80/tcp, [::]:8090->80/tcp
root-web-1
root@ip-172-31-4-171:~# docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
3e9630876ff54158ac7f65bec7010bed
root@ip-172-31-4-171:~#

```

Navigation / Tabs

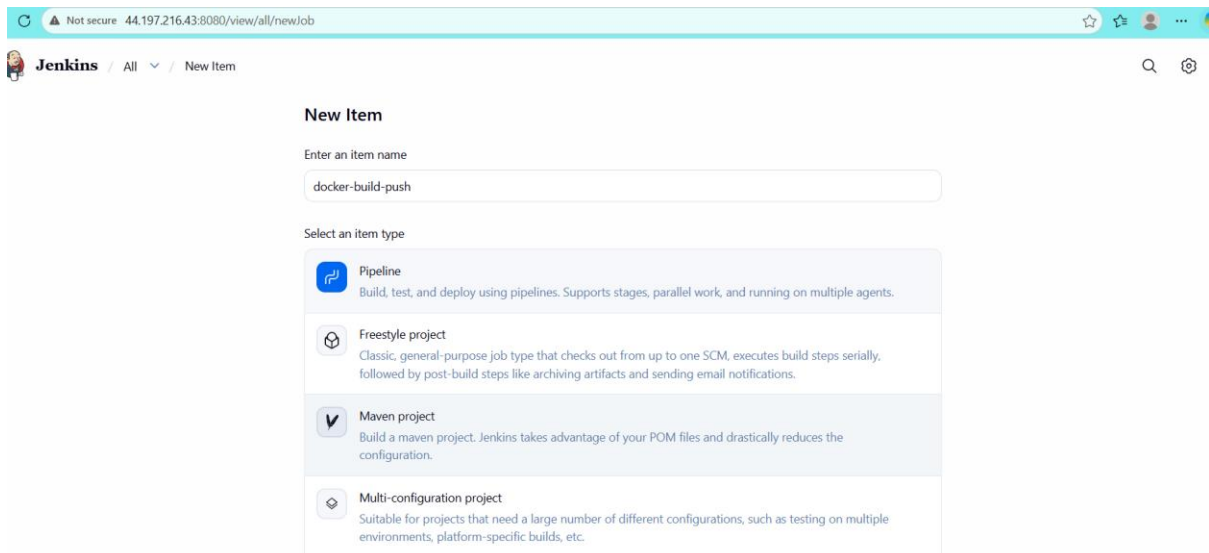
1. Open **Jenkins Dashboard**.
2. Click **New Item**.
3. Enter the item name:
docker-build-push
4. Select **Pipeline**.
5. Click **OK**.

The Pipeline job was created with the name docker-build-push.

Result

- Jenkins Pipeline job created successfully.
- Job name: **docker-build-push**

Screenshot: Jenkins → New Item → Pipeline

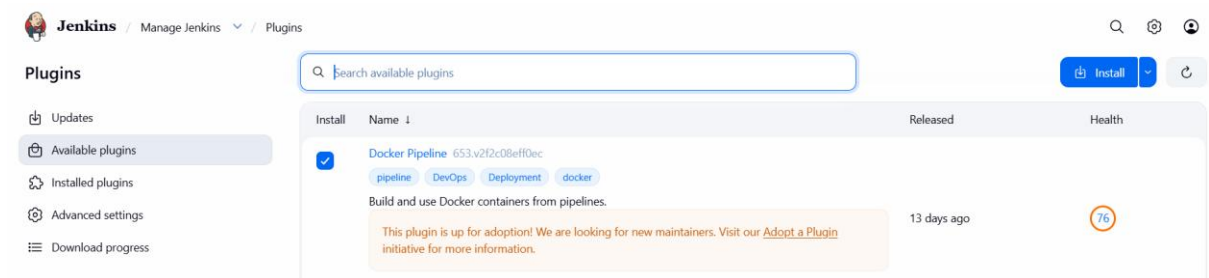


In Jenkins:

Manage Jenkins → Plugins → Available plugins → Search "Docker"

You installed:

- ✓ Docker
- ✓ Docker Pipeline



2. Configure Docker Credentials

Navigation / Tabs

1. Open **Manage Jenkins**.
2. Go to **Credentials**.
3. Select **System**.
4. Open **Global credentials**.
5. Click **Add Credentials**.

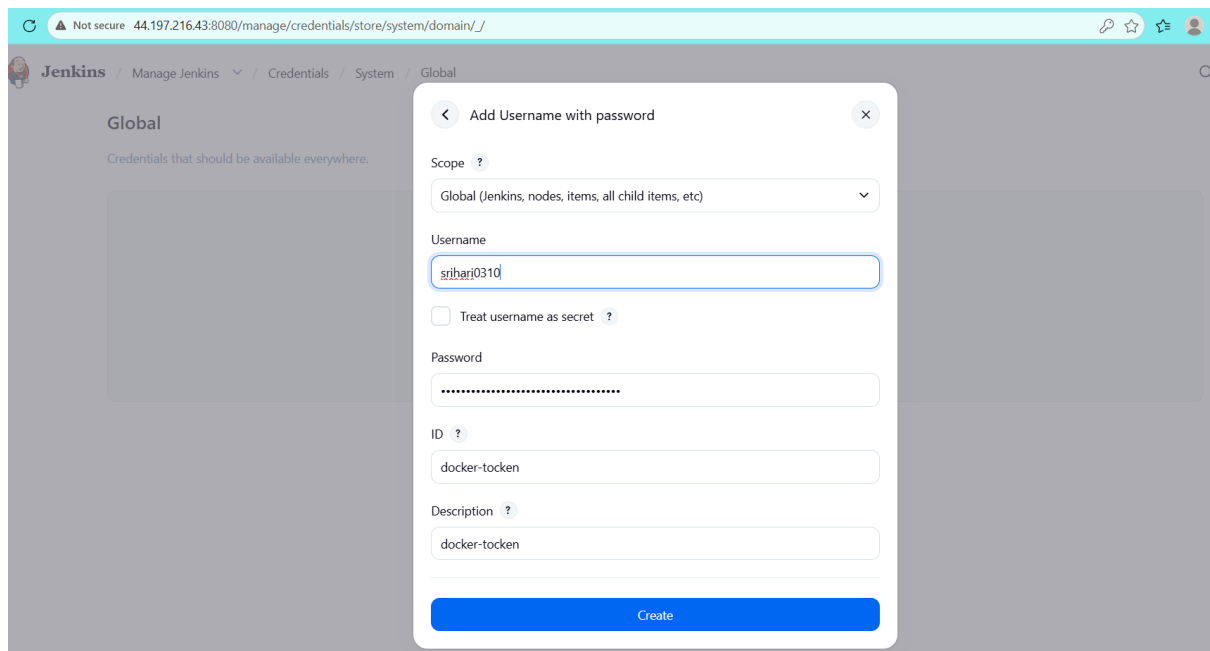
The Docker Hub credentials were configured in Jenkins Global Credentials.

Credential Configuration

- **Kind:** Username with password
- **Credentials ID:** Docker
- **Username:** Docker Hub username
- **Password:** Docker Hub password/token

The credential ID Docker is later referenced by the Jenkins Pipeline for Docker Hub authentication.

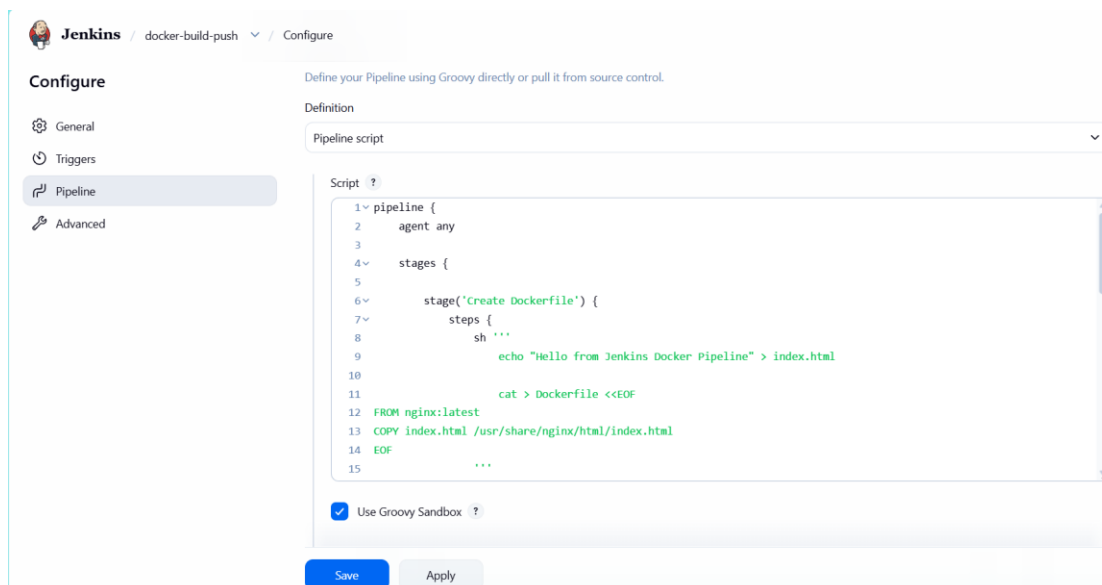
Screenshot: Manage Jenkins → Credentials → Global Credentials



3. Configure Pipeline Script

Navigation / Tabs

1. Open the **docker-build-push** Jenkins job.
2. Click **Configure**.
3. Scroll down to the **Pipeline** section.
4. In **Definition**, select **Pipeline script**.
5. Enter the Jenkins Pipeline script directly in the **Script** section.
6. Click **Save**.



Script–

```
pipeline {
```

agent any

stages {

stage('Create Dockerfile') {

steps {

sh '''

echo "Hello from Jenkins Docker Pipeline" > index.html

cat > Dockerfile <<EOF

FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html

EOF

'''

}

}

stage('Docker Build') {

steps {

sh 'docker build -t srihari0310/amazon-nginx:latest .'

}

}

stage('Docker Login') {

steps {

withCredentials([

usernamePassword{

credentialsId: 'Docker',

usernameVariable: 'DOCKER_USERNAME',

passwordVariable: 'DOCKER_PASSWORD'

)

]) {

sh '''

echo "\$DOCKER_PASSWORD" | docker login -u "\$DOCKER_USERNAME" --password-stdin

'''

}

}

}

```
stage('Docker Push') {  
    steps {  
        sh 'docker push srihari0310/amazon-nginx:latest'  
    }  
}  
}
```

PIPELINE SCRIPT EXPLANATION:

4. Create Dockerfile

Pipeline Stage: Create Dockerfile

The first stage creates an index.html file and generates a Dockerfile dynamically.

Commands Executed

```
echo "Hello from Jenkins Docker Pipeline" > index.html
```

```
cat > Dockerfile <<EOF
```

```
FROM nginx:latest
```

```
COPY index.html /usr/share/nginx/html/index.html
```

```
EOF
```

The Dockerfile uses the nginx:latest base image and copies the generated index.html into the Nginx web root.

Result

- index.html created.
 - Dockerfile created successfully.
 - Nginx configured as the base image.
-

5. Build Docker Image

Pipeline Stage: Docker Build

The Docker image was created using the following command:

```
docker build -t srihari0310/amazon-nginx:latest .
```

This stage builds the Docker image and assigns the Docker Hub repository name and tag.

Image Name

```
srihari0310/amazon-nginx:latest
```

Result

- Docker image built successfully.
 - Image tagged as srihari0310/amazon-nginx:latest.
-

6. Login to Docker Hub

Pipeline Stage: Docker Login

The Jenkins credential with ID Docker is used to authenticate with Docker Hub.

Credentials Used

Credentials ID: Docker

Username Variable: DOCKER_USERNAME

Password Variable: DOCKER_PASSWORD

The pipeline securely passes the password through docker login --password-stdin.

Command Executed

```
echo "$DOCKER_PASSWORD" | docker login -u "$DOCKER_USERNAME" --password-stdin
```

Result

- Docker Hub authentication completed successfully.
- Jenkins was able to authenticate using the configured credentials.

7. Push Docker Image to Docker Hub

Pipeline Stage: Docker Push

After successful authentication, the Docker image was pushed to Docker Hub.

Command Executed

```
docker push srihari0310/amazon-nginx:latest
```

The pipeline uses the same image name created during the Docker Build stage.

Result

- Docker image pushed successfully.
- Image available in the Docker Hub repository:

srihari0310/amazon-nginx

- Tag:

latest

8. Pipeline Execution

Navigation / Tabs

1. Open **docker-build-push**.
2. Click **Build Now**.
3. Open the generated build number.
4. Check **Stage View**.
5. Verify the status of each stage.

Pipeline Stages

Stage	Status
Create Dockerfile	SUCCESS
Docker Build	SUCCESS
Docker Login	SUCCESS

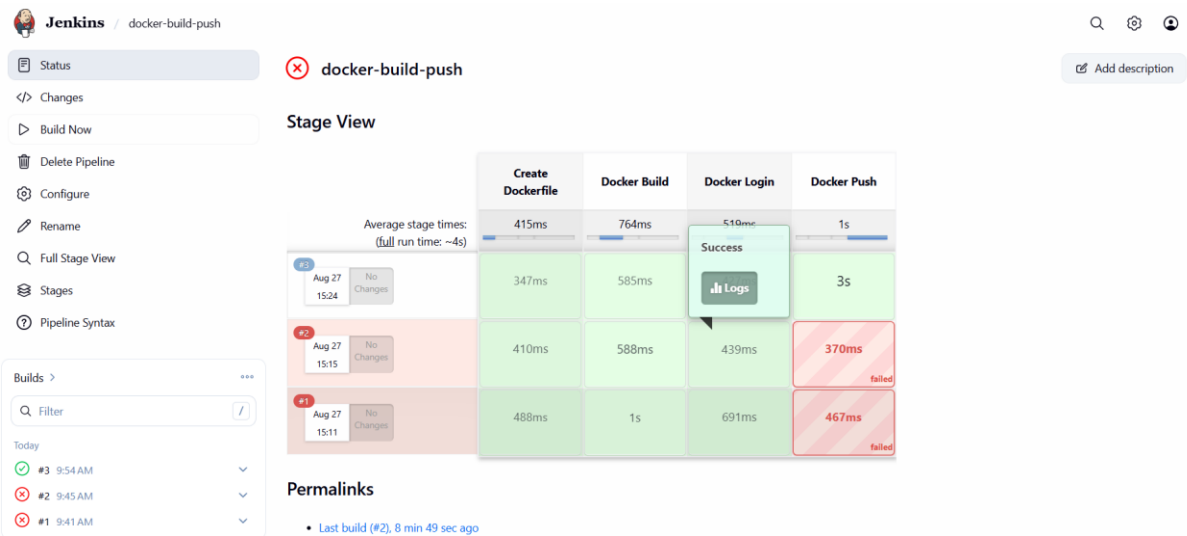
Stage	Status
Docker Push	SUCCESS

Final Result

The successful execution is visible in the Jenkins **Stage View**.

In the successful **Build #3**, all four stages are shown in green:

- **Create Dockerfile – Successful**
- **Docker Build – Successful**
- **Docker Login – Successful**
- **Docker Push – Successful**

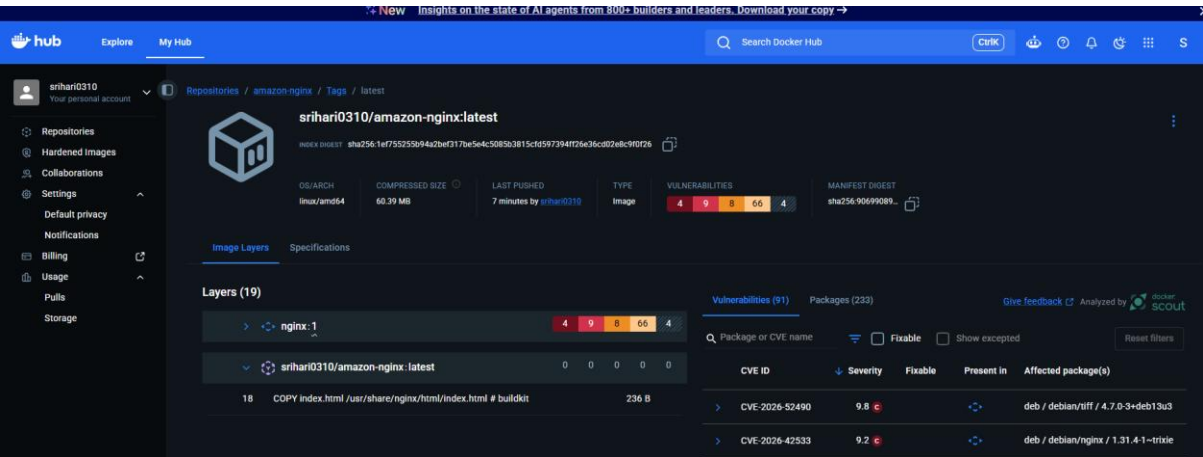


9. Final Verification

Docker Image

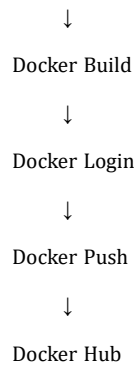
Repository: srihari0310/amazon-nginx

Tag: latest



Pipeline Flow

Create Dockerfile



Final Outcome

The Jenkins Pipeline successfully automated the complete Docker image workflow:

Dockerfile Creation → Docker Image Build → Docker Hub Authentication → Docker Image Push